

Algoritmica grafurilor - Cursul 8

27 novembre 2020

1

Fluxuri în rețele

- Drumuri de creștere
- Secțiuni
- Teorema drumului de creștere
- Teorema fluxului întreg
- Teorema flux maxim – secțiune minimă
- Algoritmul lui Ford & Fulkerson
- Algoritmul lui Edmonds & Karp

2

Exerciții pentru seminarul din săptămâna viitoare

3

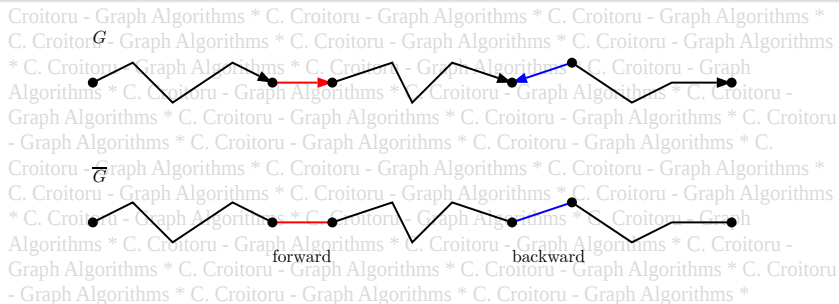
Exerciții rezolvate (parțial)

Problema fluxului maxim - Drumuri de creștere

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph

Definiție

Fie P un drum în $\tilde{G}$ – **graful suport al digrafului G** – și $e = v_i v_j$ o muchie a lui P , $e \in E(P)$. Dacă e corespunde arcului $v_i v_j$ atunci e este un **arc înainte** (forward sau direct) al lui P , altfel (e corespunde arcului $v_j v_i$) e este un **arc înapoi** (backward sau invers) al lui P .



Definiție

Fie $R = (G, s, t, c)$ o rețea și x un flux în R . Un **A-drum** (în R relativ la fluxul x) este un drum P în $\widetilde{G}$ astfel încât $\forall ij \in E(P)$:

- dacă ij este un arc direct, atunci $x_{ij} < c_{ij}$,
- dacă ij este a arc invers, atunci $x_{ji} > 0$.

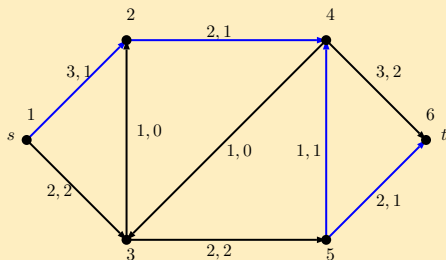
Dacă P este un A-drum în R relativ la fluxul x , atunci **capacitatea reziduală** a arcului $ij \in E(P)$ este

$$r(ij) = \begin{cases} c_{ij} - x_{ij}, & \text{dacă } ij \text{ este un arc direct al lui } P \\ x_{ji}, & \text{dacă } ij \text{ este un arc invers al lui } P \end{cases}$$

Capacitatea reziduală a drumului P este $r(P) = \min_{e \in E(P)} r(e)$.

Exemplu

Considerăm rețeaua de mai jos, unde, pe fiecare arc, prima etichetă este capacitatea iar a doua este fluxul.



$P : 1, 12, 2, 24, 4, 45, 5, 56, 6$ este un A -drum de la s la t cu arcele forward 12 ($x_{12} = 1 < c_{12} = 3$), 24 ($x_{24} = 1 < c_{24} = 2$), 56 ($x_{56} = 1 < c_{56} = 2$), și arcul backward 45 ($x_{45} = 1 > 0$). Capacitatea reziduală a lui P : $r(P) = \min \{2, 1, 1, 1\} = 1$.

Problema fluxului maxim - Drumuri de creștere

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph

Definiție

Un **drum de creștere** relativ la fluxul x în rețeaua $R = (G, s, t, c)$ este un A -drum de la s la t .

* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph

Lema 1

Dacă P este un drum de creștere relativ la fluxul x în $R = (G, s, t, c)$, atunci $x^1 = x \otimes r(P)$ definit prin

$$x_{ij}^1 = \begin{cases} x_{ij}, & \text{dacă } ij \notin E(P) \\ x_{ij} + r(P), & \text{dacă } ij \in E(P), ij \text{ este arc direct în } P \\ x_{ij} - r(P), & \text{dacă } ij \in E(P), ij \text{ este arc invers în } P \end{cases}$$

este un flux în R cu $v(x^1) = v(x) + r(P)$.

- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms

Demonstrație. Din definiția lui $r(P)$, constrângerile (i) sunt satisfăcute de x^1 . Constrângerile (ii) - verificate de x - nu sunt afectate pentru x^1 în vreun nod $i \notin V(P)$.

Pentru $i \in V(P)$ există exact două arce în P incidente cu i , e. g. li și ik . Avem următoarele două cazuri posibile:

a) li și ik sunt arce directe:

$$\begin{aligned} \sum_j x_{ji}^1 - \sum_j x_{ij}^1 &= \sum_{j \neq l} x_{ji} - \sum_{j \neq k} x_{ij} + x_{li}^1 - x_{ik}^1 \\ &= \sum_{j \neq l} x_{ji} - \sum_{j \neq k} x_{ij} + x_{li} + r(P) - x_{ik} - r(P) = \sum_j x_{ji} - \sum_j x_{ij} = 0. \end{aligned}$$

b) li este arc direct și ik arc invers:

$$\begin{aligned} \sum_j x_{ji}^1 - \sum_j x_{ij}^1 &= \sum_{j \neq l, k} x_{ji} - \sum_j x_{ij} + x_{li}^1 + x_{ki}^1 \\ &= \sum_{j \neq l, k} x_{ji} - \sum_j x_{ij} + x_{li} + r(P) + x_{ki} - r(P) = \sum_j x_{ji} - \sum_j x_{ij} = 0. \end{aligned}$$

Problema fluxului maxim - Drumuri de creștere

c) li arc invers și ik arc direct: similar cu b).

d) li și ik arce inverse: similar cu a).

$v(x^1)$ diferă de $v(x)$ datorită fluxului de pe arcul lt din P :

lt arc direct:

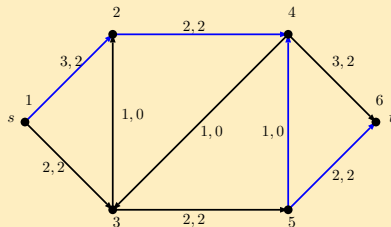
$$\begin{aligned}v(x^1) &= \sum_j x_{jt}^1 - \sum_j x_{tj}^1 = \sum_{j \neq l} x_{jt} - \sum_j x_{tj} + x_{lt}^1 = \\&= \sum_{j \neq l} x_{jt} - \sum_j x_{tj} + x_{lt} + r(P) = v(x) + r(P).\end{aligned}$$

lt arc invers:

$$\begin{aligned}v(x^1) &= \sum_j x_{jt}^1 - \sum_j x_{tj}^1 = \sum_j x_{jt} - \sum_{j \neq l} x_{tj} - x_{tl}^1 = \\&= \sum_j x_{jt} - \sum_{j \neq l} x_{tj} - (x_{tl} - r(P)) = v(x) + r(P). \quad \square\end{aligned}$$

Problema fluxului maxim - Drumuri de creștere

Pentru exemplul de mai sus, fluxul $x^1 = x \otimes r(P)$ de valoare $v(x^1) = v(x) + r(P) = 3 + 1 = 4$ este:



Remarci

- Lema de mai sus explică și numele drumurilor de creștere și capacitatea reziduală.
- Din definiție, dacă P este un drum de creștere, atunci $r(P) > 0$ și $v(x \otimes r(P)) > v(x)$. Urmează că **dacă există un drum de creștere relativ la fluxul x , atunci x nu este un flux de valoare maximă.**

Definiție

Fie $R = (G, s, t, c)$ o rețea. O **secțiune** în R este o partiție (S, T) a lui $V(G)$ cu $s \in S$ și $t \in T$. **Capacitatea secțiunii** (S, T) este

$$c(S, T) = \sum_{i \in S} \sum_{j \in T} c_{ij}.$$

Lema 2

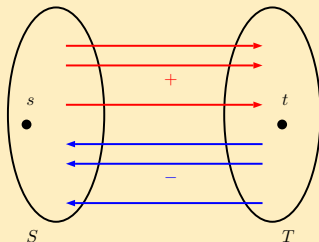
Dacă x este un flux în $R = (G, s, t, c)$ și (S, T) este o secțiune în această rețea, atunci

$$v(x) = \sum_{i \in S} \sum_{j \in T} (x_{ij} - x_{ji})$$

(valoarea fluxului este fluxul net care trece prin secțiune).

Problema fluxului maxim - Secțiuni

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms



Demonstrație.

$$\begin{aligned} v(x) &= \left(\sum_j x_{sj} - \sum_j x_{js} \right) + 0 = \\ &= \left(\sum_j x_{sj} - \sum_j x_{js} \right) + \sum_{i \in S, i \neq s} \left(\sum_j x_{ij} - \sum_j x_{ji} \right) = \end{aligned}$$

Demonstrație (continuare).

$$\begin{aligned} &= \sum_{i \in S} \left(\sum_j x_{ij} - \sum_j x_{ji} \right) = \sum_{i \in S} \sum_j (x_{ij} - x_{ji}) = \\ &= \sum_{i \in S} \sum_{j \in S} (x_{ij} - x_{ji}) + \sum_{i \in S} \sum_{j \in T} (x_{ij} - x_{ji}) = \\ &= 0 + \sum_{i \in S} \sum_{j \in T} (x_{ij} - x_{ji}). \quad \square \end{aligned}$$

Lema 3

Dacă x este un flux în $R = (G, s, t, c)$ și (S, T) este o secțiune în această rețea, atunci

$$v(x) \leq c(S, T).$$

Demonstrație. Din Lema 2

$$\begin{aligned} v(x) &= \sum_{i \in S} \sum_{j \in T} (x_{ij} - x_{ji}) \stackrel{x_{ij} \leq c_{ij}}{\leq} \sum_{i \in S} \sum_{j \in T} (c_{ij} - x_{ji}) \stackrel{x_{ji} \geq 0}{\leq} \\ &\leq \sum_{i \in S} \sum_{j \in T} c_{ij} = c(S, T). \quad \square \end{aligned}$$

Remarcă

Dacă $\bar{x}$ este un flux în R și $(\bar{S}, \bar{T})$ este o secțiune astfel încât $v(\bar{x}) = c(\bar{S}, \bar{T})$, atunci, $\forall x$ flux în R , avem $v(x) \leq c(\bar{S}, \bar{T}) = v(\bar{x})$, i. e., $\bar{x}$ este un flux de valoare maximă în R .

Similar, $\forall (S, T)$ secțiune în R , avem $c(S, T) \geq v(\bar{x}) = c(\bar{S}, \bar{T})$, adică, $(\bar{S}, \bar{T})$ este o secțiune de capacitate minimă în R .

Problema fluxului maxim - Teorema drumului de creștere

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms

Teorema 1

Un flux x este un flux de valoare maximă dacă și numai dacă nu există un drum de creștere relativ la x în R .

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms

Demonstrație. " $\Rightarrow$ " Dacă P este un drum de creștere relativ la x , atunci $x \otimes r(P)$ este un flux în R de valoare strict mai mare.

" $\Leftarrow$ " Fie x un flux în R cu proprietatea că nu există drum de creștere relativ la x în R . Fie

$$S = \{i : i \in V \text{ și } \exists P \text{ un } A\text{-drum în } R \text{ de la } s \text{ la } i\}.$$

Evident $s \in S$ (există un A -drum de lungime 0 de la s la s) și $t \notin S$ (nu există vreun A -drum de la s la t). Astfel, luând $T = V \setminus S$, (S, T) este o secțiune în R .

Problema fluxului maxim - Teorema drumului de creștere

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru

Demonstrație (continuare). $\forall i \in S$ și $\forall j \in T$ avem:

- dacă $ij \in E$, atunci $x_{ij} = c_{ij}$ și
- dacă $ji \in E$, atunci $x_{ji} = 0$

(altfel A -drumul de la s la i poate fi extins la un A -drum de la s la j , astfel $j \in S$ - contradicție). Urmează că $v(x) = \sum_{i \in S} \sum_{j \in T} (x_{ij} - x_{ji}) =$

$\sum_{i \in S} \sum_{j \in T} c_{ij} = c(S, T)$, i. e., x este un flux de valoare maximă (vezi
remarca de mai sus). $\square$

* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Problema fluxului maxim - Teorema fluxului întreg

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph

Teorema 2

(Teorema fluxului întreg) Dacă toate capacitățile din R sunt întregi atunci există un flux întreg, x , de valoare maximă (toate $x_{ij} \in \mathbb{Z}_+$).

Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -

Demonstrație. Considerăm următorul algoritm:

```
 $x^0 \leftarrow 0; i \leftarrow 0;$   
while ( $\exists P_i$  un drum de creștere relativ la  $x^i$ ) do  
     $x^{i+1} \leftarrow x^i \otimes r(P_i); i \leftarrow i + 1;$   
end while
```

Să observăm că " x^i are doar componente întregi" este un invariant al algoritmului (din definiția lui $r(P_i)$, dacă toate capacitățile sunt întregi și x^i este întreg, atunci $r(P_i)$ este întreg și astfel x^{i+1} este întreg).

- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms

Problema fluxului maxim - Teorema fluxului întreg

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru

Demonstrație (continuare). Mai mult, în fiecare iterație **while**, valoarea fluxului curent crește (cu cel puțin 1), astfel algoritmul se oprește în cel mult $c(\{s\}, V \setminus \{s\}) \in \mathbb{Z}_+$ iterații **while**.

Nu mai există drumuri de creștere relativ la fluxul final, astfel, din Teorema 1, fluxul este de valoare maximă. $\square$

* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph

Remarcă

Algoritmul de mai sus se oprește și când toate capacitățile sunt numere raționale.

* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Teorema 3

(Teorema flux maxim – secțiune minimă) Valoarea maximă a unui flux în rețeaua $R = (G, s, t, c)$ este egală cu capacitatea minimă a unei secțiuni în R .

Linia demonstrației. Dacă descriem un algoritm care, plecând cu un flux inițial x^0 (e.g., $x^0 = 0$), construiește într-un număr finit de pași un flux x fără drumuri de creștere, atunci secțiunea considerată în demonstrația Teoremei 1 satisface, împreună cu x , cerința teoremei.

Pentru capacități raționale, algoritmul considerat în demonstrația Teoremei 2 satisface această condiție. Pentru capacități arbitrare reale vom prezenta mai târziu un astfel de algoritm datorat lui **Edmonds și Karp (1972)**.

Problema fluxului maxim - Algoritmul lui Ford & Fulkerson

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -

Remarci

- O demonstrație scurtă a teoremei de mai sus constă în a arăta că există un flux de valoare maximă și de a aplica construcția din demonstrația Teoremei 1. Un flux de valoare maximă există întotdeauna observând că acesta este soluția optimă a unei probleme de programare liniară (peste un politop nevid).
- Importanța algoritmică a Teoremei 3 este dată de faptul că mulțimea tuturor secțiunilor dintr-o rețea este finită, pe când mulțimea tuturor fluxurilor este infinită.

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms

Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Problema fluxului maxim - Algoritmul lui Ford & Fulkerson

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph

Algoritmul întreține o etichetare a nodurilor din rețea pentru a determina drumuri de creștere relativ la fluxul curent x . Când nu mai există drumuri de creștere, fluxul x este de valoare maximă.

Fie $R = (G = (V, E), s, t, c)$ o rețea și x un flux în R .

Eticheta unui nod j , care are trei componente (e_1, e_2, e_3) , semnifică: există un A -drum de la s la j , P , unde $e_1 = i$ este nodul dinaintea lui j pe acest drum, $e_2 \in \{direct, invers\}$ reprezintă direcția arcului ij , iar $e_3 = r(P)$.

Inițial s este etichetat $(\cdot, \cdot, \infty)$. Celelalte noduri primesc eventual etichete prin vizitarea (scanarea) nodurilor deja etichetate:

Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Problema fluxului maxim - Algoritmul lui Ford & Fulkerson

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph

```
procedure scan(i)
for (j ∈ V, neetichetat) do
  if (ij ∈ E și  $x_{ij} < c_{ij}$ ) then
    etichetează j cu  $e = (i, direct, \min\{e_3[i], c_{ij} - x_{ij}\})$ ;
  end if
  if (ji ∈ E și  $x_{ji} > 0$ ) then
    etichetează j cu  $e = (i, invers, \min\{e_3[i], x_{ji}\})$ ;
  end if
end for
```

Semnificația componentelor etichetelor este întreținută de procedura **scan**.

Când, în procedura **scan**, nodul *t* este etichetat, se poate detecta un drum de creștere *P*, relativ la fluxul curent *x*, astfel:

Problema fluxului maxim - Algoritmul lui Ford & Fulkerson

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru

$r(P)$ este componenta e_3 a etichetei lui t , nodurile lui P pot fi găsite în $\mathcal{O}(n)$ prin explorarea primei componente a etichetelor, și modificarea $x \otimes r(P)$ se poate face în timpul acestei explorări folosind a doua componentă a etichetelor.

Pentru noul flux, se pornește cu o nouă etichetare (de la s).

Dacă toate noduri etichetate au fost scanate și nodul t nu a primit etichetă, urmează că fluxul curent nu admite drumuri de creștere, deci este de valoare maximă. Dacă S este mulțimea nodurilor etichetate și $T = V \setminus S$, atunci (S, T) este o secțiune de capacitate minimă în R .

* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Problema fluxului maxim - Algoritmul lui Ford & Fulkerson

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms

pornește cu un flux inițial $x = (x_{ij})$ (e. g., $x = 0$)

$e(s) \leftarrow (\cdot, \cdot, \infty)$;

while ($\exists$ noduri etichetate și nescanate) do

 alege i un nod etichetat și nescanat;

 scan(i);

 if (t a fost etichetat) then

 modifică fluxul pe drum dat de etichete;

 șterge toate etichetele; $e(s) \leftarrow (\cdot, \cdot, \infty)$;

 end if

end while

$S \leftarrow \{i : i \in V, i \text{ este etichetat}\}$;

$T \leftarrow V \setminus S$;

x este un flux de valoare maximă, (S, T) este o secțiune de capacitate minimă.

Problema fluxului maxim - Algoritmul lui Ford & Fulkerson

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms

Complexitatea timp: Fiecare creștere a fluxului curent necesită cel mult $2m$ ($m = |E|$) inspecții ale arcelor pentru etichetarea altor noduri. Dacă toate capacitățile sunt întregi, sunt necesare cel mult v creșteri (v fiind valoarea fluxului maxim). Astfel algoritmul are complexitatea timp $\mathcal{O}(mv)$.

Dacă U este un majorant al tuturor capacităților pe arce, atunci $v \leq (n - 1)U$ (acesta este un majorant al secțiunii $(\{s\}, V \setminus \{s\})$), deci algoritmul are complexitatea timp $\mathcal{O}(nmU)$.

Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -

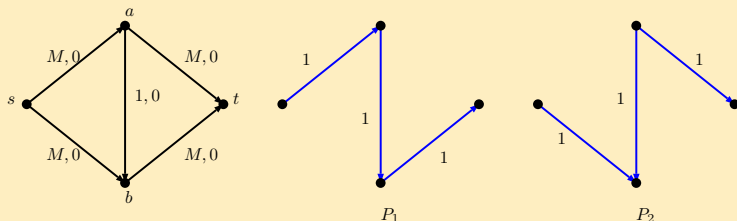
Remarcă

E posibil ca algoritmul să nu se termine pentru capacități iraționale. Această situație nu apare în implementările practice, dar neajunsul acestei descrieri a algoritmului este dat de faptul că numărul de creșteri ale fluxului curent depinde de capacități (și nu de dimensiunile rețelei).

Problema fluxului maxim - Algoritmul lui Ford & Fulkerson

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -

Exemplu



Dacă operația **alege** din algoritmul de mai sus determină drumurile de creștere $P_1, P_2, P_1, P_2, \dots$, unde $P_1 = (s, sa, a, ab, b, bt, t)$ și $P_2 = (s, sb, b, ba, a, at, t)$, atunci fiecare creștere a fluxului se face cu 1, și algoritmul necesită $2M$ creșteri, ceea ce este prea mult.

Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Aceste deficiențe ale algoritmului pot fi evitate dacă alegerea nodurilor etichetate în vederea scanării nu se face aleatoriu (Dinic (1970), Edmonds & Karp (1972)).

Definiție

Un cel mai scurt drum de creștere relativ la fluxul x în R este un drum de creștere de lungime minimă printre toate drumurile de creștere relativ la x .

Fie x un flux în rețeaua R . Fie x^k ($k \geq 1$) secvența de fluxuri:

$$x^1 \leftarrow x;$$

$$x^{k+1} \leftarrow x^k \otimes r(P_k), \quad P_k \text{ un cel mai scurt drum de creștere relativ la } x^k;$$

Modificarea lui Edmonds & Karp a algoritmului lui Ford & Fulkerson

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms

Pentru a arăta că această secvență este finită, fie $\forall i \in V$ și $\forall k \in \mathbb{N}^*$:

σ_i^k = lungimea minimă a unui A -drum de la s la i relativ la x^k .

τ_i^k = lungimea minimă a unui A -drum de la i la t relativ la x^k .

Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru

Lema 4

$\forall i \in V$ și $\forall k \in \mathbb{N}^*$ avem

$$\sigma_i^{k+1} \geq \sigma_i^k \text{ și } \tau_i^{k+1} \geq \tau_i^k.$$

Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -

Demonstrație. Omisă.

Teorema 4

(Edmonds & Karp, 1972) Dacă $x = x^1$ este un flux arbitrar în rețeaua R , atunci secvența de fluxuri $x^1, x^2, \dots$, obținută din x^1 prin creșteri succesive cu drumuri de creștere de lungime minimă, are cel mult $mn/2$ termeni (în cel mult $mn/2$ creșteri succesive se obține un flux x cu proprietatea că nu există drum de creștere relativ la x).

Demonstrație. Dacă P este un drum de creștere relativ la un flux x în R , cu capacitatea reziduală $r(P)$, un **arc critic** în P este un arc $e \in E(P)$ cu $r(e) = r(P)$. În $x \otimes r(P)$, fluxul de pe arce critice devine fie egal cu capacitatea (pe arcele directe), sau nul (pe arcele inverse).

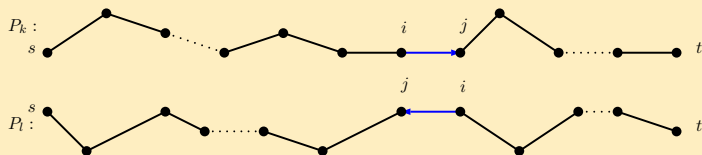
Fie ij un arc critic de pe un cel mai scurt drum de creștere P_k relativ la x^k . Lungimea lui P_k este $\sigma_i^k + \tau_i^k = \sigma_j^k + \tau_j^k$

Modificarea lui Edmonds & Karp a algoritmului lui Ford & Fulkerson

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms

Deoarece ij este critic în P_k , în x^{k+1} nu mai poate fi utilizat în aceeași direcție ca în P_k . Fie P_l (cu $l > k$) primul cel mai scurt drum de creștere relativ la x^l în care fluxul de pe arcul ij va fi modificat, (când arcul va fi folosit în direcție inversă decât în P_k). Avem două cazuri:

ij este un arc direct în P_k . Atunci $\sigma_j^k = \sigma_i^k + 1$; în P_l ij va fi arc invers, deci $\sigma_i^l = \sigma_j^l + 1$.

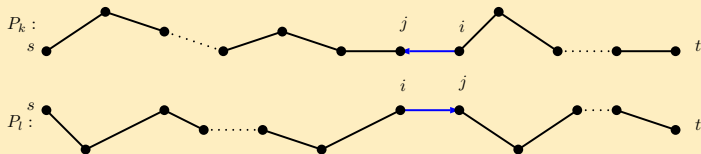


Urmează că $\sigma_i^l + \tau_i^l = \sigma_j^l + 1 + \tau_i^l \geq \sigma_j^k + 1 + \tau_i^k = \sigma_i^k + \tau_i^k + 2$ (din Lema 4). Am obținut că $length(P_l) \geq length(P_k) + 2$.

Modificarea lui Edmonds & Karp a algoritmului lui Ford & Fulkerson

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms

ij este un arc invers în P_k . Atunci $\sigma_i^k = \sigma_j^k + 1$; în P_l ij va fi arc direct, deci $\sigma_j^l = \sigma_i^l + 1$.



Urmează că $\sigma_j^l + \tau_j^l = \sigma_i^l + 1 + \tau_j^l \geq \sigma_i^k + 1 + \tau_j^k = \sigma_j^k + \tau_j^k + 2$. Obținem că $\text{length}(P_l) \geq \text{length}(P_k) + 2$.

Astfel orice cel mai scurt drum de creștere pe care arcul ij este critic are lungimea cu cel puțin 2 mai mare decât lungimea drumului precedent pe care ij a fost critic. Deoarece lungimea unui drum în G este cel mult $n - 1$, urmează că un arc fixat nu poate fi critic de mai mult de $n/2$ ori (de-a lungul întregului proces de creștere).

Modificarea lui Edmonds & Karp a algoritmului lui Ford & Fulkerson

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms

Orice drum de creștere are cel puțin un arc critic. Astfel în secvența (P_k) avem cel mult $mn/2$ drumuri de creștere cele mai scurte.

Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Corolar

În orice rețea există un flux x cu proprietatea că nu există drumuri de creștere relativ la x .

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms

Remarci

- Demonstrația Teoremei 4 este acum încheiată.
- Singura modificare din **Algoritmul lui Ford & Fulkerson** este în alegerea nodului etichetat care va fi scanat: se folosește regula "primul etichetat-primul scanat" adică, se întreține o coadă a nodurilor etichetate (inițializată cu s , la fiecare început de creștere).

Modificarea lui Edmonds & Karp a algoritmului lui Ford & Fulkerson

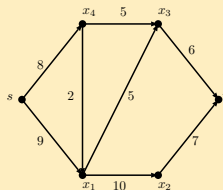
În concluzie, avem următoarea teoremă.

Teorema 5

(Edmonds-Karp, 1972) Dacă în Algoritmul lui Ford & Fulkerson scanarea noduri etichetate se face într-o manieră bfs, atunci un flux de valoare maximă se obține în $\mathcal{O}(m^2 n)$ time.

Demonstrație. Există $\mathcal{O}(mn)$ creșteri de flux (din Teorema 4), fiecare de complexitate $\mathcal{O}(m)$. $\square$

Exercițiul 1. Considerăm rețeaua de transport de mai jos:



Determinați un flux maxim în rețeaua de mai jos utilizând algoritmul lui Edmonds-Karp pentru următoarele ordonări ale listelor de adiacență:

- (a) $A(s) = (x_1, x_4)$, $A(x_1) = (x_3, x_2, x_4)$, $A(x_2) = (x_1, t)$,
 $A(x_3) = (x_1, x_4, t)$, $A(x_4) = (x_3, x_1)$.
- (b) $A(s) = (x_4, x_1)$, $A(x_1) = (x_2, x_3, x_4)$, $A(x_2) = (x_1, t)$,
 $A(x_3) = (x_1, x_4, t)$, $A(x_4) = (x_3, x_1)$.

Exerciții pentru seminarul din săptămâna viitoare

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C.

Exercițiul 2. Considerăm o rețea $R = (G, s, t, ; c)$, unde $G = (V, E)$ are n noduri și m arce, și funcția de capacitate are doar valori întregi ($c : E \rightarrow \mathbb{Z}_+$). Fie $C = \max_{e \in E} c(e)$.

- (a) Arătați că valoarea maximă a unui flux în R este cel mult $m \cdot C$.
- (b) Arătați că, pentru fiecare flux x din R și pentru fiecare $K \in \mathbb{Z}_+$, putem găsi un drum de creștere P cu capacitatea reziduală, $r(P)$, cel puțin K - dacă un asemenea drum există - în $\mathcal{O}(m)$.

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Exercițiul 2 (continuare).

(c) Considerăm următorul algoritm:

SC-MAX-FLOW(R) {

$C \leftarrow \max_{e \in E} c(e);$

$x \leftarrow 0; // x$ fluxul curent;

$K \leftarrow 2^{1 + \lfloor \log C \rfloor};$

while($K \geq 1$) {

 while(x are un drum de creștere P cu $r(P) \geq K$)

$x \leftarrow x \otimes r(P);$

$K \leftarrow K/2;$

 }

return x ;

}

Exercițiul 2 (continuare).

1. Arătați că procedura SC-MAX-FLOW(R) returnează un flux de valoare maximă x în R .
2. Arătați că, după fiecare iterație **while** exterioară, valoarea maximă a unui flux în R este cel mult $v(x) + m \cdot K$.
3. Arătați că, $\forall K \in \mathbb{Z}_+$, există cel mult $2m$ iterații **while** interioare. În consecință procedura are complexitatea timp $\mathcal{O}(m^2 \log C)$.

Exercițiul 3. Digraful $G = (V, E)$ reprezintă topologia unei rețele de procesoare. Fiecărui procesor, $v \in V$, îi cunoaștem încărcarea, $load(v) \in \mathbb{R}_+$. Folosind un flux maxim într-o anumită rețea determinați o strategie statică de echilibrare a încărcării (static load balancing strategy) în G : indicați pentru fiecare procesor încărcarea ce trebuie trimisă și cărui procesor așa încât toate procesoarele să aibă aceeași încărcare.

Exerciții pentru seminarul din săptămâna viitoare

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph

Exercițiul 4. Fie $R = (G, s, t, c)$ o rețea și $(S_i, T_i) (i = \overline{1, 2})$ două secțiuni de capacitate minimă în R . Arătați că $(S_1 \cup S_2, T_1 \cap T_2)$ și $(S_1 \cap S_2, T_1 \cup T_2)$ sunt de asemenea secțiuni de capacitate minimă.

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph

Exercițiul 5. Fie $R = (G = (V, E), s, t, c)$ o rețea și $c : E \rightarrow \mathbb{Z}_+$, $n = |V|$, $m = |E|$.

- (a) Descrieți un algoritm de complexitate timp $\mathcal{O}(n + m)$ pentru a determina o secțiune de capacitate minimă în R , având la îndemână un flux de valoare maximă x^* .
- (b) Folosind un algoritm de flux maxim pentru o anumită funcție de capacitate, arătați că se poate găsi o secțiune de capacitate minimă în R , (S_0, T_0) , cu număr minim de arce.

- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Exerciții pentru seminarul din săptămâna viitoare

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C.

Exercițiul 6. Fie $G = (S, T; E)$ un graf bipartit. Demonstrați teorema lui Hall (*există un cuplaj în G care saturează toate nodurile din S dacă și numai dacă $(H) \forall A \subseteq S, |N_G(A)| \geq |A|$*) folosind **teorema flux maxim - secțiune minimă** pe o rețea particulară.

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms

Exercițiul 7. Fie $R = (G, s, t, c)$ o rețea care are toate capacitățile întregi pozitive și pare. Arătați că există un flux maxim cu toate valorile întregi pozitive și pare.

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Exercițiul 8. Fie $R = (G, s, t, c)$ o rețea astfel ca $st, ts \notin E(G)$. Avem și o funcție majorant $m : E(G) \rightarrow \mathbb{R}_+$, $m(e) \leq c(e), \forall e \in E(G)$. Un flux φ în R se numește flux legal în R dacă $x(e) \geq m(e), \forall e \in E(G)$.

(a) Arătați că, pentru orice flux legal φ și orice st -secțiune (S, T) avem

$$v(\varphi) \leq \sum_{i \in S, j \in T, ij \in E(G)} c(ij) - \sum_{i \in S, j \in T, ji \in E(G)} m(ji)$$

(b) Pornind de la R construim o rețea $\bar{R} = (\bar{G}, \bar{s}, \bar{t}, \bar{c})$, unde

- $V(\bar{G}) = V(G) \cup \{\bar{s}, \bar{t}\}$ (acestea sunt noduri noi);
- $E(\bar{G}) = E(G) \cup \{st, ts\} \cup \{\bar{s}v, v\bar{t} : v \in V(G)\}$;
- pentru orice $v \in V(G)$, $\bar{c}(\bar{s}v) = \sum_{uv \in E(G)} m(uv)$ și $\bar{c}(v\bar{t}) =$

$$\sum_{vu \in E(G)} m(vu);$$

- $\bar{c}(st) = \bar{c}(ts) = \infty$ și pentru orice $ij \in E(G)$, $\bar{c}(ij) = c(ij) - m(ij)$.

Exercițiul 8 (continuare).

Arătați că există un flux legal în R dacă și numai dacă există un flux în $\bar{R}$ de valoare $M = \sum_{e \in E(G)} m(e)$.

- (c) Folosind un flux legal inițial *legal flux* (vezi b)) arătați Ford&Fulkum se poate modifica algoritmul Ford-Fulkerson pentru a obține un flux legal de valoare maximă.

Exercițiul 9. Fie $R = (G, s, t, c)$ o rețea de transport.

- (a) Descrieți un algoritm eficient care să verifice dacă o tăietură dată, (S, T) , este de capacitate minimă în R .
- (b) Descrieți un algoritm de complexitate timp $\mathcal{O}(n + m)$ care să verifice dacă un flux dat, x , este de valoare maximă în R .
- (c) Fie $c : E \rightarrow \mathbb{Z}_+$ și x^* este un flux întreg de valoare maximă în R . Să presupunem că pe un arc, $e_0 \in E(G)$, capacitatea crește cu 1. Descrieți un algoritm eficient care să determine un flux maxim în noua rețea

Exercițiul 10. Fie x un flux întreg într-o rețea dată $R = (G, s, t, c)$, cu $v(x) = v_1 + v_2 + \dots + v_p$, unde $v_i \in \mathbb{N}^*$, $\forall i = \overline{1, p}$, $p \geq 1$. Arătați că există p fluxuri întregi $x^1, x^2, \dots, x^p$ astfel ca $v(x^i) = v_i$, $\forall i = \overline{1, p}$ și

$$\mathbf{x} = \sum_{i=1}^p \mathbf{x}^i.$$

Exercițiul 1. Soluție.

(a) Valoarea unui flux x în R poate fi majorată astfel:

$$v(X) = \sum_{su \in E} x(su) - \sum_{vs \in E} x(vs) \leq \sum_{su \in E} x(su) \leq \sum_{su \in E} c(su) \leq m \cdot C$$

(b) Dacă un astfel de drum există, poate fi găsit folosind algoritmul Ford-Fulkerson pentru determinarea drumurilor de creștere, dar cu următoarele restricții (de ce?):

- pentru un arc $uv \in E$, dacă u este etichetat, v poate fi și el etichetat (dacă nu este deja) numai dacă $c(uv) > x(uv) + K$;
 - pentru un arc $vu \in E$, dacă u este etichetat, v poate fi și el etichetat (dacă nu este deja) numai dacă $x(vu) > K$;
- În acest fel capacitatea reziduală de pe fiecare arc eligibil este cel puțin K (de ce?). Inspectarea tuturor acestor arce necesită timpul $\mathcal{O}(m)$.

Exerciții rezolvate (parțial)

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph

- (c1) An drum de creștere în R nu poate avea o capacitate reziduală strict mai mare decât C ; cum $2^{1+\lceil \log C \rceil} \geq C$, procedura de mai sus detectează toate drumurile de creștere relativ la x , deci la final x este un flux maxim.
- (c2) Să presupunem că, fluxul curent x , nu admite drumuri de creștere P de capacitate reziduală $\delta(P) \geq K$ (aceasta se întâmplă după o iterație în *while*-ul exterior).
- Dacă începem o nouă procedură de etichetare ca în (b) vom termina cu o tăietură (S, T) în care toate nodurile etichetate sunt în S , $s \in S, t \in T$ și:
 - $\forall uv \in E$, if $u \in S$ și $v \in T$, atunci $c(uv) \leq x(uv) + K$ (de ce?);
 - $\forall vu \in E$, if $u \in S$ și $v \in T$, atunci $x(vu) \leq K$ (de ce?);

Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Exerciții rezolvate (parțial)

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph

- Pe de altă parte, v_0 , valoarea maximă a unui flux în R este:

$$v_0 \leq c(S, T) = \sum_{\substack{u \in S, v \in T, \\ uv \in E}} c(uv) \leq \sum_{\substack{u \in S, v \in T, \\ uv \in E}} [x(uv) + K] =$$

$$\stackrel{\text{why?}}{=} v(x) + \sum_{\substack{u \in S, v \in T, \\ uv \in E}} K + \sum_{\substack{u \in S, v \in T, \\ vu \in E}} x(vu) \leq$$

$$\leq v(x) + \sum_{\substack{u \in S, v \in T, \\ uv \in E}} K + \sum_{\substack{u \in S, v \in T, \\ vu \in E}} K \leq v(x) + m \cdot K$$

- (c3) Fie x fluxul curent, înaintea intrării într-o nouă iterație a **while**-ului exterior pentru valoarea curentă alui K .

Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Exerciții rezolvate (parțial)

- Din (c2) știm că următoarea iterație în bucla **while** exterioară va crește fluxul cu cel mult $2m \cdot K$.
- Fiecare iterație din bucla **while** interioară va crește valoarea acestui flux cu cel puțin K ; astfel, vor fi cel mult $2m$ iterații în bucla **while** interioară.
- Există $(2 + \lfloor \log C \rfloor)$ iterații în bucla **while** exterioară.
- Deci, complexitatea timp totală este $\mathcal{O}(m^2 \log C)$ (de ce?).

Croitoru - Graph Algorithms ^ C. Croitoru - Graph Algorithms ^ C. Croitoru - Graph Algorithms ^
C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms ^ C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Exercițiul 3. Soluție. Fie m încărcarea medie totală,

$$m = \frac{1}{n} \sum_{v \in V} \text{load}(v) \text{ și } V_{<} = \{v \in V : \text{load}(v) < m\},$$

$$V_{=} = \{v \in V : \text{load}(v) = m\}, \quad V_{>} = \{v \in V : \text{load}(v) > m\}$$

- $(V_{<}, V_{=}, V_{>})$ este o partiție (slabă) a lui V .
- Adăugăm două noduri noi s, t (sursa și destinația fictive) și definim un digraf $G' = (V', E')$, unde $V' = V \cup \{s, t\}$, $E' = E \cup \{sv : v \in V_{>}\} \cup \{ut : u \in V_{<}\}$.
- Definim deasemeni capacitatea arcelor:

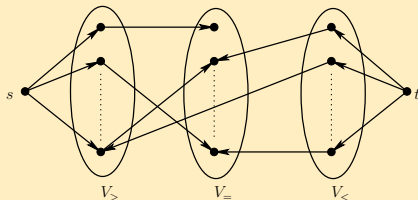
$$c(sv) = \text{load}(v) - m, \quad \forall v \in V_{>},$$

$$c(ut) = m - \text{load}(u), \quad \forall u \in V_{<},$$

$$c(xy) = +\infty, \quad \forall xy \in E.$$

Exerciții rezolvate (parțial)

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -



- Putem să arătăm că există o strategie de echilibrare statică dacă și numai dacă rețeaua $R = (G', s, t, c)$ are un flux φ care saturează toate arcele care pleacă din s (sau echivalent, toate arcele care intră în t - **de ce?**).

Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

- " $\implies$ " Să presupunem că știm pentru orice $uv \in E$ încărcarea $\varphi(uv)$ care trebuie transferată de la u către v .
- Definim $\varphi(sv) = c(sv), \forall v \in V_>$ și $\varphi(ut) = c(ut), \forall u \in V_<$.
- Evident $0 \leq \varphi(xy) \leq c(xy), \forall xy \in E'$; pentru un procesor $v \in V_>$ supra-încărcarea $load(v) - m$ va părăsi v , deci

$$\varphi(sv) = load(v) - m = \sum_{vx \in E} \varphi(vx) - \sum_{yv \in E} \varphi(yv)$$

Un argument similar pentru orice $u \in V_<$:

$$\varphi(ut) = m - load(u) = \sum_{xu \in E} \varphi(xu) - \sum_{uy \in E} \varphi(uy)$$

Aceste două relații arată că legea de conservare a fluxului este respectată.

- " $\Leftarrow$ " Dacă φ este un flux în R , atunci din legea de conservare orice procesor supra-încărcat își va descărca această supra-încărcare către celelalte procesoare iar orice procesor sub-încărcat va primi încărcarea necesară până la m de la vecinii săi.
- **Care poate fi ordinea de transfer a încărcării între procesoare?** (Cu alte cuvinte cum se poate evita situația în care $load(uv)$ este mai mare decât încărcarea curentă din u ?)

Exercițiul 4. Soluție.

- Arătăm că $(S_1 \cup S_2, T_1 \cap T_2)$ este o tăietură de capacitate minimă; evident că este o bipartiție a lui $V(G)$ (de ce?), și $s \in S_1 \cup S_2, t \in T_1 \cap T_2$.
- Fie x un flux maxim în R ($c(S_1, T_1) = c(S_2, T_2) = v(x)$), avem

$$c(uv) = x(uv), x(vu) = 0, \forall u \in S_i, v \in T_i., \text{ (de ce?)}$$

- Astfel,

$$c(uv) = x(uv), x(vu) = 0, \forall u \in S_1 \cup S_2, \forall v \in T_1 \cap T_2$$

$$\Rightarrow c(S_1 \cup S_2, T_1 \cap T_2) = v(x)$$

- Deci $(S_1 \cup S_2, T_1 \cap T_2)$ este deasemeni o tăietură de capacitate minimă (de ce?).

Exercițiul 5. Soluție.

(b) Vom folosi notația $|(S, T)| := |\{uv \in E(G) : u \in S, v \in T\}|$, pentru orice tăietură (S, T) .

- Fie $R' = (G, s, t, c')$ o rețea cu $c'(uv) = (|E(G)| + 1)c(uv) + 1$. În acest fel, pentru orice tăietură (S, T) we have $c'(S, T) = (|E(G)| + 1) \cdot c(S, T) + |(S, T)|$.
- Suppose now that (S_0, T_0) și (S, T) are two cuts of G , (S_0, T_0) of capacitate minimă tăietură in R' :

$$c'(S_0, T_0) = (|E(G)| + 1) \cdot c(S_0, T_0) + |(S_0, T_0)| \stackrel{\text{why?}}{\leq}$$

$$\leq c'(S, T) = (|E(G)| + 1) \cdot c(S, T) + |(S, T)| \implies$$

$$\implies (|E(G)| + 1) \cdot [c(S_0, T_0) - c(S, T)] + |(S_0, T_0)| \stackrel{\text{why?}}{\leq} |(S, T)|$$

Exerciții rezolvate (parțial)

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -

- Să presupunem că $c(S_0, T_0) \geq c(S, T) + 1$, atunci

$$|E(G)| + 1 + |(S_0, T_0)| \leq |(S, T)| \leq |E(G)| - \text{contradicție.}$$

- Dacă (S_0, T_0) este o tăietură de capacitate minimă în R' și (S, T) de capacitate minimă în R avem $c(S_0, T_0) = c(S, T)$ și

$$c'(S_0, T_0) = (|E(G)| + 1) \cdot c(S_0, T_0) + |(S_0, T_0)| \stackrel{\text{dece?}}{\leq}$$

$$\leq (|E(G)| + 1) \cdot c(S_0, T_0) + |(S, T)|,$$

astfel, $|(S_0, T_0)| \leq |(S, T)|$.

Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Exercițiul 6. Soluție. Fie $R = (H, s, t, c)$ următoarea rețea:

$$V(H) = V(G) \cup \{s, t\}, \quad E(H) = \{su : u \in S\} \cup \{uv : uv \in E(G)\} \cup \\ \cup \{vt : v \in T\},$$

$$c(su) = 1, \forall u \in S; \quad c(uv) = 1, \forall uv \in E(G); \quad c(vt) = 1, \forall v \in T.$$

- Evident, oricărui cuplaj M din G îi corespunde un flux întreg x_M în R :

$$x_M(uv) = 1, \text{ dacă } uv \in M; \quad x_M(su) = 1, \text{ dacă } u \in S(M),$$

$$x_M(vt) = 1, \text{ dacă } v \in S(M) \text{ și } x_M(xy) = 0 \text{ pe orice arc } xy.$$

- Reciproc, oricărui flux întreg x din R îi putem asocia un cuplaj M_x of G :

$$M_x = \{uv \in E(G) : x(uv) = 1, u \in S \text{ și } v \in T\}.$$

- Mai mult, $|M| = v(x_M)$.
- Dacă $M \in \mathcal{M}_G$ saturează toate nodurile din S ($S \subseteq S(M)$), atunci x_M va fi un flux maxim în R (de ce?).
- Fie $A \subseteq S$ și $(\Sigma, \Theta) = (A \cup N_G(A) \cup \{s\}, V(G) \setminus S)$,

$$c(\Sigma, \Theta) = \sum_{v \in N_G(A)} c(vt) + \sum_{u \in S \setminus A} c(su) \geq v(x_M), \forall A \subseteq S$$

ceea ce este echivalent cu (de ce?):

$$|N_G(A)| + |S \setminus A| \geq |S|, \forall A \subseteq S \Leftrightarrow (H) : |N_G(A)| \geq |A|, \forall A \subseteq S.$$

Exerciții rezolvate (parțial)

- Dacă (H) are loc, atunci fie (Σ, Θ) o tăietură de capacitate minimă din R .
- Fie $A = \Sigma \cap S$ și $B = \Sigma \cap T$.
- $B \subseteq N_G(A)$ (de ce?)
- Pentru un nod $v \in N_G(A) \setminus B$, avem $c(\Sigma \cup \{v\}, \Theta \setminus \{v\}) = c(\Sigma, \Theta) - |N_G(v) \cap A| + 1 \leq c(\Sigma, \Theta)$.
- Astfel putem presupune că $N_G(A) = B$.
- Pentru un flux maxim întreg x :

$$\begin{aligned} |M_x| = v(x) &= c(\Sigma, \Theta) = |S \setminus A| + |N_G(A)| \\ &\geq |S \setminus A| + |A| = |S| \Rightarrow |M_x| = |S|, \text{ deci } S \subseteq S(M_x). \end{aligned}$$

Algorithms * C. Croitoru Graph Algorithms * C. Croitoru Graph Algorithms * C. Croitoru

Exercițiul 7. Soluție.

- Se folosește algoritmul Edmonds-Karp plecând cu un flux inițial

Exercițiul 9. Soluție.

(a) Fie (S, T) o tăietură și x un flux în R ; avem

$$c(S, T) = \sum_{i \in S} \sum_{j \in T} c_{ij} \geq \sum_{i \in S} \sum_{j \in T} x_{ij} \geq \sum_{i \in S} \sum_{j \in T} (x_{ij} - x_{ji}) = v(x).$$

- Dacă x este flux de valoare maximă, atunci (S, T) is de capacitate minimă dacă și numai dacă $c(S, T) = v(x)$ ceea ce este echivalent cu $c_{ij} = x_{ij}, \forall ij \in E(G), i \in S, j \in T$ (de ce?).
- Mai întâi se determină un flux de cost maxim în the rețea și apoi se verifică relațiile de mai sus.

- (b) Se încearcă determinarea unui nou drum de creștere relativ la x - fluxul dat.
- (c) Se încearcă determinarea unui nou drum de creștere relativ la x^* - fluxul maxim dat.

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -

Exercițiul 10. Soluție. Demonstrăm prin inducție după $v(x)$.

- Pentru $v(x) = 0$ proprietatea e evidentă.
- Fie $v(x) > 0$ și $x' : E(G) \rightarrow \mathbb{N}$ un flux întreg astfel încât $x'(e) \leq x(e)$, pentru orice $e \in E(G)$, $v(x') \geq v_1$ iar suma $\sum_{e \in E(G)} x'(e)$ este minimă (de ce există un astfel de flux?).
- Arătăm că $v(x') = v_1$.
- Presupunem prin reducere la absurd că $v(x') \neq v_1$ și fie $S = \{u \in V(G) : \exists P \in \mathcal{P}_{su} \text{ a. î. } x'(e) > 0, \forall e \in E(P)\}$.
- t trebuie să fie în S (de ce?).

Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Exerciții rezolvate (parțial)

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru

- Există un drum $P \in \mathcal{P}_{st}$ cu $x'(e) > 0, \forall e \in E(P)$.
- Scăzând cu 1 fluxul de pe arcele acestui drum obținem un flux x'' cu $x''(e) \leq x(e)$, pentru orice $e \in E(G)$, $v(x'') = v(x') - 1 \geq v_1$ (de ce?), și $\sum_{e \in E(G)} x''(e) < \sum_{e \in E(G)} x'(e)$, contradicție.
- Astfel, $x - x'$ is a flux de valoare $\sum_{i=2}^p v_i$ și putem aplica ipoteza inductivă.

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *